Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Prácticas Iniciales — Sección C

MANUAL TÉCNICO

Backend — Servidor REST API con Node.js y Supabase

Carnet: 202406087
Fecha: 19/03/2026

1. Descripción General del Proyecto

Este manual describe la arquitectura, tecnologías y endpoints del servidor backend desarrollado para la Práctica de Desarrollo Web. El sistema permite a los estudiantes de la Facultad de Ingeniería calificar y comentar sobre catedráticos y cursos del área de Ciencias y Sistemas.

El servidor sigue una arquitectura cliente-servidor, donde el backend expone una REST API consumida por el frontend desarrollado en React.

2. Tecnologías Utilizadas

Node.js

Entorno de ejecución de JavaScript del lado del servidor. Permite crear el servidor HTTP y manejar múltiples solicitudes de forma asíncrona.

Express.js

Framework minimalista para Node.js que facilita la creación de rutas, middleware y manejo de peticiones HTTP.

PostgreSQL / Supabase

Supabase es una plataforma de base de datos en la nube basada en PostgreSQL. Se utiliza para almacenar toda la información de usuarios, publicaciones, comentarios, cursos y catedráticos.

Librerías principales

- express — Servidor y enrutamiento HTTP
- pg — Conector de PostgreSQL para Node.js
- dotenv — Carga de variables de entorno desde archivo .env
- bcryptjs — Encriptación segura de contraseñas
- jsonwebtoken — Generación y verificación de tokens JWT
- cors — Permite peticiones desde el frontend en otro dominio/puerto

3. Estructura del Proyecto

```
backend/
├── src/
│   ├── controlador/          # Lógica de negocio de cada recurso
│   │   ├── authController.js
│   │   ├── publicacionController.js
│   │   ├── comentarioController.js
│   │   ├── usuarioController.js
│   │   └── catedraticoCursoController.js
│   ├── rutas/                # Definición de endpoints
│   │   ├── authRoutes.js
│   │   ├── publicacionRoutes.js
│   │   ├── comentarioRoutes.js
│   │   ├── usuarioRoutes.js
│   │   └── catedraticoCursoRoutes.js
│   ├── intermediario/        # Middleware de autenticación
│   │   └── authMiddleware.js
│   └── db.js                 # Configuración de conexión a Supabase
├── .env                      # Variables de entorno (no subir a GitHub)
├── .gitignore
├── index.js                  # Punto de entrada del servidor
└── package.json
```

4. Configuración del Entorno

Variables de entorno (.env)

El archivo .env contiene las credenciales sensibles del proyecto y no debe subirse al repositorio:

```
PORT=3000
DATABASE_URL=postgresql://usuario:password@host:6543/postgres
JWT_SECRET=clave_secreta_para_firmar_tokens
```

Conexión a la base de datos (src/db.js)

Se utiliza un Pool de conexiones de la librería pg, que mantiene conexiones reutilizables hacia Supabase, mejorando el rendimiento del servidor. El parámetro ssl: { rejectUnauthorized: false } es requerido para conexiones a Supabase.

5. Sistema de Autenticación

El sistema usa JWT (JSON Web Tokens) para autenticar usuarios. Al hacer login, el servidor genera un token firmado con JWT_SECRET que el cliente debe enviar en el header de cada petición protegida:

```
Authorization: Bearer <token>
```

El middleware authMiddleware.js verifica este token antes de permitir el acceso a rutas protegidas. Si el token es inválido o no existe, responde con error 401.

Las contraseñas se encriptan con bcryptjs usando un factor de costo de 10 antes de almacenarse en la base de datos, por lo que nunca se guarda la contraseña en texto plano.

6. Endpoints del API REST

Base URL: http://localhost:3000/api

Formato de todas las respuestas: JSON

Autenticación (/api/auth)

Registrar usuario

Método	Detalle
Método	**POST**
URL	`/api/auth/registro`
Auth	No requerido
Descripción	Registra un nuevo usuario en el sistema. Verifica que el correo y registro académico no estén en uso, encripta la contraseña y guarda el usuario.
Body (JSON)	`{ "id_usuario": "USR001", "registro_academico": "202406087", "nombres": "Oscar", "apellidos": "Gomez", "correo": "oscar@gmail.com", "contrasena": "123456" }`
Respuesta	`{ "mensaje": "Usuario registrado correctamente" }`

Iniciar sesión

Método	Detalle
Método	**POST**
URL	`/api/auth/login`
Auth	No requerido
Descripción	Autentica al usuario con su registro académico y contraseña. Si las credenciales son correctas, retorna un token JWT válido por 8 horas.
Body (JSON)	`{ "registro_academico": "202406087", "contrasena": "123456" }`
Respuesta	`{ "mensaje": "Login exitoso", "token": "eyJhbGci...", "usuario": { "id_usuario": "USR001", "nombres": "Oscar" } }`

Recuperar contraseña

Método	Detalle
Método	**POST**
URL	`/api/auth/recuperar-contrasena`
Auth	No requerido
Descripción	Permite restablecer la contraseña verificando que el registro académico y correo coincidan en la base de datos.
Body (JSON)	`{ "registro_academico": "202406087", "correo": "oscar@gmail.com", "nueva_contrasena": "nueva123" }`
Respuesta	`{ "mensaje": "Contrasena actualizada correctamente" }`

Publicaciones (/api/publicaciones)

Obtener todas las publicaciones

Método	Detalle
Método	**GET**
URL	`/api/publicaciones`
Auth	No requerido
Descripción	Retorna todas las publicaciones ordenadas por fecha descendente, con datos del usuario, curso y catedrático asociados.
Respuesta	`[ { "id_publicacion": "PUB001", "mensaje": "...", "fecha": "2026-03-19", "nombres": "Oscar", ... } ]`

Obtener publicaciones por curso

Método	Detalle
Método	**GET**
URL	`/api/publicaciones/curso/:id_curso`
Auth	No requerido
Descripción	Filtra las publicaciones por el ID de un curso específico.
Respuesta	`[ { "id_publicacion": "PUB001", "mensaje": "...", "nombre_curso": "Estructuras de Datos" } ]`

Obtener publicaciones por catedrático

Método	Detalle
Método	**GET**
URL	`/api/publicaciones/catedratico/:id_catedratico`
Auth	No requerido
Descripción	Filtra las publicaciones por el ID de un catedrático específico.
Respuesta	`[ { "id_publicacion": "PUB001", "mensaje": "...", "nombre_catedratico": "Carlos" } ]`

Crear publicación

Método	Detalle
Método	**POST**
URL	`/api/publicaciones`
Auth	JWT requerido
Descripción	Crea una nueva publicación asociada al usuario autenticado. El id_usuario se obtiene del token JWT, no del body.
Body (JSON)	`{ "id_publicacion": "PUB001", "id_curso": "CUR001", "id_catedratico": "CAT001", "mensaje": "Excelente catedratico" }`
Respuesta	`{ "mensaje": "Publicacion creada correctamente" }`

Comentarios (/api/comentarios)

Obtener comentarios de una publicación

Método	Detalle
Método	**GET**
URL	`/api/comentarios/:id_publicacion`
Auth	No requerido
Descripción	Retorna todos los comentarios de una publicación específica, con el nombre del usuario que comentó.
Respuesta	`[ { "id_comentario": "COM001", "mensaje": "...", "nombres": "Oscar", "fecha": "2026-03-19" } ]`

Crear comentario

Método	Detalle
Método	**POST**
URL	`/api/comentarios/:id_publicacion`
Auth	JWT requerido
Descripción	Agrega un nuevo comentario a una publicación. El id_usuario se obtiene del token JWT.
Body (JSON)	`{ "id_comentario": "COM001", "mensaje": "Estoy de acuerdo" }`
Respuesta	`{ "mensaje": "Comentario creado correctamente" }`

Usuarios (/api/usuarios)

Obtener perfil de usuario

Método	Detalle
Método	**GET**
URL	`/api/usuarios/:id_usuario`
Auth	No requerido
Descripción	Retorna la información pública de un usuario: id, registro académico, nombres, apellidos y correo.
Respuesta	`{ "id_usuario": "USR001", "registro_academico": "202406087", "nombres": "Oscar", "apellidos": "Gomez", "correo": "oscar@gmail.com" }`

Actualizar perfil

Método	Detalle
Método	**PUT**
URL	`/api/usuarios/:id_usuario`
Auth	JWT requerido
Descripción	Permite al usuario autenticado actualizar sus datos personales (nombres, apellidos, correo). El registro académico no puede modificarse.
Body (JSON)	`{ "nombres": "Oscar Alberto", "apellidos": "Gomez", "correo": "nuevo@gmail.com" }`
Respuesta	`{ "mensaje": "Perfil actualizado correctamente" }`

Obtener cursos aprobados

Método	Detalle
Método	**GET**
URL	`/api/usuarios/:id_usuario/cursos-aprobados`
Auth	No requerido
Descripción	Retorna los cursos aprobados por el usuario y el total de créditos acumulados.
Respuesta	`{ "cursos": [ { "nombre_curso": "Estructuras de Datos", "creditos": 4 } ], "total_creditos": 4 }`

Cursos y Catedráticos

Obtener todos los catedráticos

Método	Detalle
Método	**GET**
URL	`/api/catedraticos`
Auth	No requerido
Descripción	Retorna la lista completa de catedráticos registrados en el sistema.
Respuesta	`[ { "id_catedratico": "CAT001", "nombres": "Carlos", "apellidos": "Mendez", "correo": "..." } ]`

Obtener todos los cursos

Método	Detalle
Método	**GET**
URL	`/api/cursos`
Auth	No requerido
Descripción	Retorna la lista completa de cursos del área de sistemas registrados en el sistema.
Respuesta	`[ { "id_curso": "CUR001", "nombre_curso": "Estructuras de Datos", "creditos": 4, "area": "Sistemas" } ]`

7. Estructura de la Base de Datos

La base de datos está alojada en Supabase (PostgreSQL) y contiene las siguientes tablas:

- Usuario — Almacena los datos de los usuarios registrados
- Catedratico — Información de los catedráticos de la facultad

- Curso — Cursos del área de Ciencias y Sistemas
- Publicacion — Publicaciones sobre cursos o catedráticos
- Comentario — Comentarios en respuesta a publicaciones
- Curso_Aprobado — Relación entre usuarios y cursos que han aprobado